📚 1. Numeric Types

These store numbers.

🔹 a) int – Integer (whole numbers)

```
x = 10
print(type(x))  # <class 'int'>
```

🔹 b) float – Floating-point (decimal numbers)

```
y = 10.5
print(type(y))  # <class 'float'>
```

🔹 c) complex – Complex numbers (a + bj)

```
z = 3 + 5j
print(type(z))  # <class 'complex'>
```

📜 2. String (str)

Used for text data, enclosed in quotes.

```
name = "Python"
print(type(name))  # <class 'str'>
```

You can use **single**, **double**, or **triple quotes**.

📋 3. Sequence Types

These store multiple items.

🔹 a) list – Ordered and changeable

```
fruits = ["apple", "banana", "cherry"]
print(type(fruits))  # <class 'list'>
```

🔹 b) tuple – Ordered but unchangeable (immutable)

```
numbers = (1, 2, 3)
```

```
print(type(numbers))  # <class 'tuple'>
```

🔹 **c) range – Sequence of numbers**

```
r = range(5)
print(list(r))  # [0, 1, 2, 3, 4]
```

🗺️ **4. Mapping Type**

🔹 **dict – Key-value pairs**

```
person = {"name": "John", "age": 25}
print(type(person))  # <class 'dict'>
```

Access by key:

```
print(person["name"])  # John
```

🧺 **5. Set Types**

Used for storing **unique, unordered** values.

🔹 **a) set**

```
colors = {"red", "green", "blue"}
print(type(colors))  # <class 'set'>
```

🔹 **b) frozenset – Immutable set**

```
fs = frozenset({"apple", "banana"})
print(type(fs))  # <class 'frozenset'>
```

✅ **6. Boolean Type**

Represents True or False.

```
is_active = True
print(type(is_active))  # <class 'bool'>
```

🪄 7. None Type

Represents **no value** or **null**.

```
x = None

print(type(x))  # <class 'NoneType'>
```

⚙️ Summary Table

Data Type	Example	Description
int	10	Integer numbers
float	3.14	Decimal numbers
complex	2 + 3j	Complex numbers
str	"Hello"	Text data
list	[1, 2, 3]	Ordered, changeable
tuple	(1, 2, 3)	Ordered, unchangeable
range	range(5)	Sequence of numbers
dict	{"a":1}	Key-value pairs
set	{"a", "b"}	Unique values
frozenset	frozenset({"a", "b"})	Immutable set
bool	True	Boolean values
NoneType	None	No value

Python Number Types

Python has **three main numeric data types**:

Type	Description	Example
int	Integer (whole numbers)	10, -5, 1000
float	Floating-point (decimal numbers)	3.14, -2.5, 0.0
complex	Complex numbers (with real + imaginary parts)	2 + 3j, -5j

---

🔹 1. Integer (int)

Represents **whole numbers**, both positive and negative.

```
x = 10
y = -25
z = 0

print(x, type(x))  # 10 <class 'int'>
print(y, type(y))  # -25 <class 'int'>
print(z, type(z))  # 0 <class 'int'>
```

✅ No decimal point
✅ No limit on size (Python automatically manages big integers)

🔹 2. Floating Point (float)

Represents **decimal (real) numbers**.

```
a = 3.14
b = -0.5
c = 2.0

print(a, type(a))  # 3.14 <class 'float'>
print(b, type(b))  # -0.5 <class 'float'>
```

```
print(c, type(c))  # 2.0 <class 'float'>
```

✅ Includes decimal point
✅ Used for scientific and financial calculations

🔹 **3. Complex Numbers (complex)**

Represent numbers with **real** and **imaginary** parts.

Format:

a + bj

where a is the real part and b is the imaginary part.

```
x = 2 + 3j
y = 5 - 4j


print(x, type(x))  # (2+3j) <class 'complex'>
print(x.real)  # 2.0
print(x.imag)  # 3.0
```

✅ Imaginary part is written with **j** (not i as in math)
✅ Python supports arithmetic on complex numbers